

清华大学本科生考试试题专页纸	
考试课程： 操作系统（A）	时间：2025年06月13日上午 09:00~12:00

系别：	班级：	学号：	姓名：
-----	-----	-----	-----

答卷注意 事项：	1. 答题前，请先在试题纸和答卷本上写明A卷或B卷、系别、班级、学号和姓名。
	2. 在答卷本上答题时，要写明题号，不必抄题。
	3. 试卷有20道判断题、7道问答题。
	4. 请回答所有题目。本试卷共8页。最后空白页可做草稿。
	5. 考试完毕，必须将试题纸和答卷本一起交回。

一、(20分) 判断题

- 1. ☐ 硬链接文件可以跨文件系统，而软链接文件不能跨文件系统。
- 2. ☐ 在虚拟内存管理中，进程发生缺页异常时，操作系统终止当前进程以释放内存资源。
- 3. ☐ 在Linux中，多个进程同时读一个文件时，Linux会在该文件inode中以互斥访问的方式记录该文件的访问时间。
- 4. ☐ 信号量初始值为1时，可用于实现互斥；初始值为0时，可用于实现同步。
- 5. ☐ 管程（Monitor）的条件变量signal操作在Hoare语义下立即切换线程，让发起signal操作的线程睡眠，让被唤醒线程执行。
- 6. ☐ 日志文件系统通过先将元数据修改写入日志区域再提交到实际位置，可确保文件系统崩溃后通过重放日志恢复一致性，但无法防止因存储设备硬件故障导致的数据块损坏。
- 7. ☐ 父进程退出后，子进程的资源会得不到释放并变成僵尸进程，所以在操作系统设计中要确保父进程的退出发生在子进程退出之后。
- 8. ☐ 临界区的访问规则包括让权等待、空闲则入、有限等待、忙则睡眠。
- 9. ☐ 目录是一种特殊文件，其内容由<该目录下文件名，指向文件元数据的指针>项组成的列表。
- 10. ☐ 进程间直接通信的方式包括：信号、管道、共享内存。
- 11. ☐ 同一进程中的线程A可直接修改同一进程的线程B的栈上的变量。
- 12. ☐ 出现死锁的必要条件包括互斥、持有并等待、非抢占和按序等待。
- 13. ☐ 在单处理器系统中，通过禁用中断可以实现临界区的互斥访问；但在多处理器系统中，仅禁用中断无法保证互斥。
- 14. ☐ 银行家算法可以判断系统是否处于死锁状态。
- 15. ☐ 信号量机制中，P操作和V操作既能在同一进程内的多线程中使用，也能在不同进程之间使用。
- 16. ☐ 条件变量必须与互斥锁配合使用，因wait操作需原子性释放锁并进入等待状态。
- 17. ☐ 采用分页内存管理机制可消除内存的外碎片和内碎片问题。
- 18. ☐ 在管程（Monitor）中，若线程在等待条件变量时不释放锁，则会引发死锁。。
- 19. ☐ 若资源分配图中存在环路且每类资源仅有一个实例，则系统必然死锁。
- 20. ☐ 进程关闭或删除无名管道(pipe)的过程中，涉及进程管理和内存管理，与具体的文件系统无关。

二、(80分)问答题

1. (4分)

mmap系统调用可以将文件内容映射至内存地址空间中，这样可以通过访问内存的方式随机访问文件内容。

```
void *mmap(void *addr, size_t len, int prot, int flags, int fd, off_t offset);
```

参数说明：

addr：建议映射的起始地址（通常设为 NULL，由内核选择）。

len：映射区域的长度（需按页对齐）。

prot：内存保护权限（位掩码组合）：

- PROT_READ：可读
- PROT_WRITE：可写
- PROT_EXEC：可执行
- PROT_NONE：不可访问

flags：映射类型（关键标志）：

- MAP_SHARED：修改同步到文件，多进程共享
- MAP_PRIVATE：写时复制（COW），修改不写回文件
- MAP_ANONYMOUS：匿名映射（忽略 fd 和 offset）
- MAP_FIXED：强制使用 addr 地址（若冲突则覆盖原有映射）

fd：文件描述符（匿名映射时设为 -1）

offset：文件映射的起始偏移量（需按页对齐）

请分析采用mmap系统调用方式的文件访问相比于基于read/write系统调用的文件访问的优势。（回答内容小于50个字）

2. (6分)

设存在一个基于索引方式的文件系统，可支持硬链接和软链接。

- 2.1 设文件索引节点中有6个地址项，其中3个地址项是直接地址索引，2个地址项是一级间接地址索引，1个地址项是二级间接地址索引，每个地址项大小为4字节。若磁盘索引块和磁盘数据块大小均为128字节，则可表示的单个文件最大长度是多少？请给出计算过程。（回答内容小于50个字）
- 2.2 设文件F1的当前引用计数值为1，用户通过命令行进行相关文件操作，先建立F1的符号链接（软链接）文件F2，再建立F1的硬链接文件F3，再建立F2的硬链接文件F4，然后删除F1和F2。此时，F3和F4的引用计数值分别是多少？请给出计算过程。（回答内容小于50个字）

3. (6分)

某采用rcore或ucore操作系统的机器人有 9 个机械臂，由 K 个进程竞争使用，每个进程最多需要控制 4 个机械臂。

- 3.1 该机器人可能会发生死锁的 K 的最小值是多少？请给出计算过程和简明的解释。（回答内容小于40个字）
- 3.2 如果该机器人采用如下的死锁检测算法detect_deadlock来发现死锁问题，请基于进程数n和资源类型数m给出死锁检测算法的时间复杂度（大O符号表示法），并给出对你答案的简要说明。（回答内容小于60个字）

```

1 // 输入:
2 // Allocation:  $n \times m$  矩阵, Allocation[i][j] 表示进程 i 当前持有资源 j 的数量
3 // Available: 长度为 m 的向量, Available[j] 表示资源 j 的当前可用数量
4 // Request:  $n \times m$  矩阵, Request[i][j] 表示进程 i 当前请求的资源 j 的数量
5 // n: 进程数量
6 // m: 资源类型数量
7 // 输出:
8 // deadlocked_processes: 死锁进程的集合 (未标记的进程)
9
10 function detect_deadlock(Allocation, Available, Request, n, m):
11     // 步骤 1: 初始化标记数组和临时向量
12     let marked[1..n] = false // 标记进程是否非死锁, 初始全为 false (未标记)
13     let W[1..m] = copy of Available // 临时向量 W 初始化为 Available
14
15     // 步骤 2: 标记无资源分配的进程 (Allocation 行全零)
16     for i = 1 to n:
17         if sum(Allocation[i]) == 0: // 检查进程 i 的 Allocation 行是否全零
18             marked[i] = true // 标记为 true (非死锁)
19
20     // 步骤 3: 查找可标记的进程
21     let found = true
22     while found:
23         found = false
24         for i = 1 to n:
25             if not marked[i]:
26                 // 检查 Request[i] ≤ W
27                 // (即 Request[i][k] ≤ W[k] for all k=1 to m)
28                 can_mark = true
29                 for k = 1 to m:
30                     if Request[i][k] > W[k]:
31                         can_mark = false
32                         break // 退出内层循环
33                 if can_mark:
34                     // 标记进程 i 并更新 W
35                     marked[i] = true
36                     found = true
37                     for k = 1 to m:
38                         W[k] = W[k] + Allocation[i][k] // 释放进程 i 的资源
39
40     // 步骤 4: 收集死锁进程 (未标记的进程)
41     deadlocked_processes = []
42     for i = 1 to n:
43         if not marked[i]:
44             deadlocked_processes.append(i) // 添加死锁进程
45
46     return deadlocked_processes

```

4. (16分)

小朱学习了TestAndSet原子操作指令后, 准备用它进行实战。小朱设计了基于栈的生产者-消费者模型, 即若干生产者和若干消费者线程需要一个线程安全的结构进行push操作 (生产者行为) 和pop操作 (消费者行为), 小朱用类C语言编写了如下伪代码来满足同步互斥的生产者消费者行为, 请阅读代码并回答:

```

1 struct Node {
2     Node *next;

```

```

3     int value;
4 };
5
6     struct Stack {
7         Node *top;           // 成员变量栈顶
8         _____ // 成员变量锁 添加代码 1
9     };
10
11    void init(Stack *s) {
12        s->top = NULL;
13        _____ // 初始化锁 添加代码 2
14    }
15 }
16
17 void push(Stack *s, Node *n) {
18     while (1) {
19         _____ // 准备进入临界区 添加代码 3
20         Node *old_top = s->top;
21         n->next = old_top;
22         s->top = n;
23         _____ // 离开临界区 添加代码 4
24         return;
25     }
26 }
27
28 Node *pop(Stack *s) {
29     while (1) {
30         _____ // 添加代码 5
31         Node *old_top = s->top;
32         if (old_top == NULL){
33             _____ // 添加代码 6
34             return NULL;
35         }
36         Node *new_top = old_top->next;
37         s->top = new_top;
38         _____ // 添加代码 7
39         return old_top;
40     }
41 }

```

4.1 下面是unlock操作的伪代码，请给出基于TestAndSet原子操作指令的lock操作的伪代码函数实现和必要的注释。（注释内容小于40个字）

unlock操作

```

typedef struct { int flag; // 锁标志: 0 = 未锁定, 1 = 已锁定 } Lock;
Void unlock(Lock* lock) {    lock.flag = 0; // 解锁    }

```

4.2 请在“_____”处的代码1 -- 7，填写Lock相关的类C伪代码（可直接使用lock()和unlock()函数），以使得上述代码符合小朱的需求。可添加C伪代码的注释。（每处代码的注释内容小于20个字）

5. (18分)

下面是用类C语言实现的Peterson算法，但存在错误：

```

1 // 共享变量
2 bool flag[2] = {false, false}; // 标识线程是否请求进入临界区
3 int turn = 0;                    // 记录轮次
4

```

```

5 // 线程 0 的入口函数
6 void* thread0(void* arg) {
7     while (true) {
8         flag[0] = true;           // 步骤 1: 声明自己请求进入临界区
9         turn = 1;                 // 步骤 2: 将轮次让给线程 1
10        while (turn == 0 && flag[1]); // 步骤 3: 循环等待条件
11        // 临界区开始
12        /* 执行临界区任务 */
13        // 临界区结束
14        flag[0] = false;          // 步骤 4: 释放请求
15    }
16    return NULL;
17 }
18
19 // 线程 1 的入口函数 (与线程 0 对称)
20 void* thread1(void* arg) {
21     while (true) {
22         flag[1] = true;
23         turn = 0;
24         while (turn == 1 && flag[0]);
25         /* 执行临界区任务 */
26         flag[1] = false;
27     }
28     return NULL;
29 }

```

- 5.1 指出上述代码会破坏哪些临界区访问规则，并给出一个具体的执行序列说明互斥性如何被破坏。（回答内容小于100个字）
- 5.2 请说明如何修复代码中的错误，确保满足同步互斥的必要条件。（回答内容小于50个字）
- 5.3 如果扩展到多个线程，可以采用面包师同步互斥算法（Bakery Algorithm）。请分析下面的面包师同步互斥算法代码是否满足同步互斥的必要条件。如果你认为满足，请解释该算法：a) 如何保证互斥的；b) 如何保证没有饥饿现象；c) 如何保证没有死锁（回答内容小于150个字）。如果你认为不满足，请说明如何修复代码中的错误，确保满足同步互斥的必要条件（回答内容小于150个字）。

```

1 // 共享变量声明
2 boolean choosing[n] = {false}; // 标记线程是否正在取号
3 int number[n] = {0};           // 存储每个线程的排队号码
4
5 // 线程 i 的代码
6 while (true) {
7     // 1. 取号阶段
8     choosing[i] = true;           // 标记正在取号
9     number[i] = max(number[0..n-1]) + 1; // 获取当前最大号码+1
10    choosing[i] = false;          // 取号完成
11
12    // 2. 等待阶段：检查所有其他线程
13    for (int j = 0; j < n; j++) {
14        if (j == i) continue; // 跳过自身
15
16        // 等待线程 j 完成取号
17        while (choosing[j]) { /* 忙等待 */ }
18
19        // 等待条件：
20        // - 线程 j 有有效号码 (number[j] ≠ 0)
21        // - 且 (j 的号码更小) 或 (号码相同但 j 的线程 ID 更小)
22        while (number[j] != 0 &&
23                (number[j] < number[i] ||
24                 (number[j] == number[i] && j < i))) {

```

```

25      /* 忙等待 */
26      }
27  }
28
29      // 3. 进入临界区
30      // ... 执行临界区代码 ...
31
32      // 4. 退出临界区
33      number[i] = 0; // 重置号码, 释放资源
34
35      // 剩余代码 (非临界区)
36  }

```

6. (22分)

理发店有1位理发师、1把理发椅和5把供等候理发的顾客坐的椅子。规则如下：

- 没有顾客时，理发师在理发椅上等待顾客。
- 顾客到来时，若理发师在睡觉，则叫醒理发师。
- 理发师忙碌时，顾客在空椅上等待；无空椅则顾客离开。
- 理发师每次只服务一位顾客。

下面是设计的理发师同步互斥问题的伪代码实现。信号量的定义如下：

- barber_ready: 理发师就绪信号量，表示理发师是否准备好服务
- customer_ready: 顾客就绪信号量，表示有顾客等待理发
- mutex: 互斥锁信号量，保护共享变量waiting_customers
- barber_chair: 理发椅互斥锁信号量，确保同一时间仅一位顾客用理发椅
- waiting_customers (共享变量): 当前等待的顾客数

6.1 信号量定义、初始化和P操作伪码如下，请给信号量P操作和V操作的类C伪码实现。

```

typedef struct {
    int value; // 资源计数器
    WaitQueue q; // 等待队列
} semaphore;

semaphore sema = 0; //表示 sema.value = 0
P(sema); // sema信号量的P操作    V(sema); // sema信号量的V操作

```

6.2 请参考已填好的代码1，填写”_____”的代码2-17的内容，保证理发师同步互斥行为的正确性。

```

1  // 信号量定义
2  semaphore barber_ready = _____ 0 _____; //用于参考: 设初值 代码 1
3  semaphore customer_ready = _____; //设初值 填写代码 2
4  semaphore mutex = _____; //设初值 填写代码 3
5  semaphore barber_chair = _____; //设初值 填写代码 4
6  int waiting_customers = _____; //设初值 填写代码 5
7  void barber() { // 理发师线程
8      while (true) {
9          P(_____); // 等待顾客 填写代码 6
10         P(_____); //填写代码 7
11         waiting_customers--; // 减少等待顾客数
12         V(_____); //填写代码 8
13         V(_____); // 通知顾客准备就绪 填写代码 9
14         cut_hair(); // 理发(耗时操作)
15         V(_____); // 释放理发椅 填写代码 10
16     } // end while
17 } // end barber
18 void customer() { // 顾客线程
19     P(_____); //填写代码 11
20     if ( _____ ) { // 如果无空椅填写代码 12
21         V(_____); //填写代码 13

```

```

22     leave();           // 离开
23 } else {
24     waiting_customers++;
25     V( );              //填写代码 14
26     V( );              // 唤醒理发师 填写代码 15
27     P( );              // 等待理发师准备填写代码 16
28     P( );              // 占用理发椅填写代码 17
29     get_haircut();      // 接受理发服务
30     // 注：顾客离开时无需操作 barber_chair（由理发师释放）
31 } //end if
32 } // end customer

```

7. (8分)

请回答有关日志文件系统的下列问题。

7.1 日志文件系统的主要目的是什么？（回答内容小于40个字）

7.2 考虑以下simple-journal-fs的伪代码。请回答问题：（a）假设系统在journal_commit()完成后、checkpoint()开始前崩溃。请说明恢复过程应如何处理该事务？（回答内容小于50个字）（b）如果崩溃发生在journal_write()期间（部分日志块未持久化），请说明恢复过程应如何确保一致性？（回答内容小于50个字）

```

1  // 事务结构
2  struct transaction {
3      int num_blocks;           // 块数量
4      int block_ids[MAX_BLOCKS]; // 磁盘块 ID
5      char *data[MAX_BLOCKS];   // 数据
6  };
7
8  void journal_write(transaction *tx) {
9      // 步骤 1: 将事务写入日志区
10     for (int i = 0; i < tx->num_blocks; i++) {
11         disk_write(LOG_BASE + i, tx->data[i]); // 写入日志块
12     }
13     disk_flush(); // 持久化日志写入
14 }
15
16 void journal_commit(transaction *tx) {
17     // 步骤 2: 写入提交记录
18     disk_write(LOG_COMMIT, COMMIT_RECORD); // 提交块（如 TxE）
19     disk_flush(); // 持久化提交
20 }
21
22 void checkpoint(transaction *tx) {
23     // 步骤 3: 数据写入最终位置
24     for (int i = 0; i < tx->num_blocks; i++) {
25         disk_write(tx->block_ids[i], tx->data[i]); // 写入目标块
26     }
27     disk_flush();
28 }
29
30 void journal_free() {
31     // 步骤 4: 释放日志空间
32     disk_write(LOG_SUPERBLOCK, FREE_RECORD); // 更新日志超级块
33 }

```

